

Seminar 8: PL/SQL SUBPROGRAMS

-Procedures and functions-

- Subprogram = a set of statements seen as a unit which has a name and makes a certain processing.
- The subprogram is called from another program or subprogram, written in PL/SQL or in an universal programming language.
- *When defining* the subprogram the following elements have to be described:
 - The header (contains the type of the subprogram, its name, the parameters)
 - The body (contains a set of statements)
 - The end (an instruction that is meant to exit the subprogram and return to the main program)
- The call of the subprogram contains its name and the actual parameters.
- The parameters can be: IN, OUT or IN OUT. When defining them their precision is not mentioned. When calling the subprogram their order has to be maintained (also their type and number).
- In the body of subprograms, global or substitution variables cannot be used. They can be transmitted through parameters.
- In order to display the errors SHOW ERRORS command can be used.

The formal parameters can be:

- **IN** (the value of the actual parameter is transferred in the variable that is defined in the procedure. This variable is read-only.) The actual parameter never modifies. It is the default type;
- **OUT** (the value of the formal parameter is transferred in the actual parameter when the subprogram ends);
- **IN OUT** (the values are transferred from one type of variable to the other, when executing/ending the subprogram).
- `:= IN_parameter`
- `OUT_parameter :=`
- `IN_OUT_parameter :=` or `:= IN_OUT_parameter`

1. PROCEDURES

```
CREATE [OR REPLACE] PROCEDURE procedure_name  
[(param1 [IN|OUT|IN OUT] type1,  
param2[IN|OUT|IN OUT] type2, ....) ]
```

```
IS|AS
```

```
-- declaring the local variables in the procedure
```

```
-- it is used instead of DECLARE
```

```
BEGIN
```

```
----
```

```
[EXCEPTION]
```

```
-----
```

```
END [procedure_name];
```

Deleting the procedure:

DROP PROCEDURE *procedure_name*;

Calling a procedure:

- In an anonymous PL/SQL block or in another subprogram;
- Directly using **EXECUTE** *procedure_name* or **EXEC** *procedure_name* or **CALL** *procedure_name*.
 - EXEC/EXECUTE is a SQL*PLUS command that puts the subprogram in an anonymous block BEGIN... END. Usually, a call that has been made using EXEC consumes less memory than a CALL command. Obs: SQL*PLUS is an Oracle interface, not to be confused with SQL.
 - CALL is a SQL command and can use only SQL data types (pay attention to Boolean). Has the INTO clause, which can be used to specify a host variable for the value returned by a function. CALL can inhibit the exception's propagation to the environment. CALL can be used in triggers instead of their bodies, in order to call another procedure.
- From another Oracle (APEX, Oracle Developer Suite) or non-Oracle environment.

Procedure with IN parameters

Example: Create a procedure that modifies the salary of an employee whose id is given as parameter. Also the percentage of salary growth is given as a parameter.

CREATE OR REPLACE

PROCEDURE modify_salary

(v_employee_id IN employees.employee_id%type, percent number)

IS

v_salary employees.salary%type;

BEGIN

Select salary into v_salary from employees where employee_id=v_employee_id;

dbms_output.put_line('The employee has the salary:'||v_salary);

Update employees

Set salary=salary(1+percent/100)*

Where employee_id=v_employee_id;

Select salary into v_salary from employees where employee_id=v_employee_id;

dbms_output.put_line('The employee has the salary:'||v_salary);

END;

/

show errors;

The possible calls:

CALL modify_salary(176, 10)

or

EXECUTE modify_salary(176, 10)

or (equivalent with the previous call)

begin

modify_salary(176, 10);

end;

Procedure with OUT parametres

Example: Create a procedure that returns the name and the salary of an employee whose id is given.

```
CREATE OR REPLACE PROCEDURE proc_employee
(p_employee_id IN employees.employee_id%type,
name OUT employees.first_name%type,
p_salary OUT employees.salary%type)
IS
BEGIN
Select first_name,salary into name, p_salary from employees where
employee_id=p_employee_id;
END;
```

The call:

```
DECLARE
name employees.first_name%type;
v_salary employees.salary%type;
BEGIN
proc_employee(150, name, v_salary);
DBMS_OUTPUT.PUT_LINE(' the employee '||name||' has the salary: '||v_salary);
END;
```

Example: Create a procedure that calculates the average salary and returns it. Use a host variable.

```
CREATE or REPLACE PROCEDURE avg_sal
(p_avg OUT number)
IS
BEGIN
Select AVG(salary) into p_avg from employees;
END;
```

The call:

```
VARIABLE v_sal NUMBER
EXECUTE avg_sal(:v_sal)
Print v_sal
```

Procedure with IN OUT parameters

Example: Create a procedure that modifies the salary of all employees if their salary is smaller than the average salary. The procedure receives the average salary and returns it after the updates have been made. It calls the modify_salary procedure and uses 15 for the percentage.

```
CREATE or REPLACE PROCEDURE modify_avg_salary
(avg_salary IN OUT number)
IS
```

```
cursor c is select employee_id, salary from employees where salary<avg_salary;  
BEGIN  
for var in c loop  
modify_salary(var.employee_id,15);  
end loop;  
select avg(salary) into avg_salary from employees;  
End;
```

The call:

```
DECLARE  
v_avg_salary number;  
BEGIN  
select avg(salary) into v_avg_salary from employees;  
dbms_output.put_line('The initial average salary: '||round(v_avg_salary));  
modify_avg_salary(v_avg_salary);  
dbms_output.put_line('The final average salary: '||round(v_avg_salary));  
end;
```

2. FUNCTIONS

```
CREATE [OR REPLACE] FUNCTION function_name  
[(param1 [IN/OUT/IN OUT] type1,  
param2[IN/OUT/IN OUT] type2, ....) ]  
RETURN type  
IS|AS  
--  
BEGIN  
----  
RETURN value;  
[EXCEPTION]  
-----  
END [function_name];
```

For viewing the returned type:

```
DESCRIBE function_name;
```

For deleting the function:

```
DROP FUNCTION function_name;
```

Example: Create a function that returns TRUE/FALSE if the employee whose id is given has the salary higher/smaller than the average. The function returns NULL if the employee doesn't exist.

```
CREATE OR REPLACE FUNCTION check_salary  
(p_employee_id IN employees.employee_id%type)
```

RETURN Boolean

IS

v_salary employees.employee_id%type;

average_salary number;

BEGIN

select avg(salary) into average_salary from employees;

dbms_output.put_line('avg sal: '||average_salary);

SELECT salary into v_salary from employees where employee_id=p_employee_id;

dbms_output.put_line('salary: '||v_salary);

IF v_salary > average_salary then

return true;

ELSE

return false;

end if;

EXCEPTION

WHEN no_data_found THEN

return NULL;

end;

Display the returned type:

describe check_salary;

Call the function for the following employees:

11, 104, 110

BEGIN

if check_salary(11) then

dbms_output.put_line('the salary is higher');

elsif (not check_salary(11)) then

dbms_output.put_line('the salary is smaller');

else

dbms_output.put_line('the employee doesn't exist');

end if;

end;